

Санкт-Петербургский государственный университет

Кафедра системного программирования

Группа 22.Б07-мм

Разработка системы кэширования метаданных в программно-определяемой СХД

Шалашнов Егор Дмитриевич

Отчёт по учебной практике
в форме «Решение»

Научный руководитель:
инженер-исследователь лаборатории технологий программирования инфраструктурных
решений СПбГУ, Васенина А. И.

Санкт-Петербург
2025

Оглавление

Введение	3
1. Постановка задачи	4
2. Обзор	5
2.1. Блочный уровень ядра Linux	7
2.2. Системы кэширования на блочном уровне	8
2.3. Выводы	10
3. Описание решения	11
3.1. Внесенные изменения	11
3.2. Жизненный цикл кэш-линии	12
3.3. Политика вытеснения	14
3.4. Обработка поступающих запросов	14
4. Тестирование	16
Заключение	18
Список литературы	19

Введение

Экспоненциальный рост объёмов данных и необходимость высокой производительности предъявляют к системам хранения данных (СХД) всё более высокие требования. Современные СХД должны эффективно использовать дисковое пространство, выдерживать смешанные нагрузки чтения/записи, быстро восстанавливаться после сбоев. Для достижения этих целей производители внедряют механизмы моментальных снимков, сборки мусора, репликации, дедупликации, компрессии и многое другое.

Одним из широко применяемых подходов в современных СХД является лог-структурированная организация записи. Новые данные и обновления, не перезаписывают существующие блоки, а размещаются в новых областях, что упрощает построение снимков и повышает целостность. Лог-структурированность применяется в ряде решений — от файловых систем (F2FS [4], SpriteLFS [7], ...), до баз данных (ApacheCassandra[1]) и блочных уровней в программно-определяемых СХД.

Лог-структурированность уже реализована в текущей версии продукта компании Shvacher, однако в нынешней реализации все необходимые метаданные хранятся в оперативной памяти. Такой подход дает минимальные задержки обращения, однако ограничивает масштабируемость по объёму и числу используемых томов, повышается риск давления на память и деградации производительности.

Для более рационального использования ресурсов было принято решение внедрить систему кэширования метаданных, которая удерживает наиболее часто используемые записи в оперативной памяти, а ”холодные” — размещает на диске с возможностью подкачки по требованию.

В данной работе описывается разработка и исследование прототипа такой системы кэширования метаданных для СХД с закрытым исходным кодом, разрабатываемой компанией Shvacher.

1. Постановка задачи

Целью данной работы является разработка прототипа системы кэширования метаданных, предназначенного для использования в программно определяемой СХД, разрабатываемой компанией Shvacher.

Для её выполнения были поставлены следующие задачи:

1. Провести обзор существующих решений.
2. Разработать прототип системы кэширования.
3. Интегрировать прототип, проверив соответствие ожидаемому поведению и отсутствие нарушения существующей логики.

2. Обзор

В контексте данной работы, **кэш** — это компонент, который осуществляет посредничество при обмене данными между инициатором I/O (например, файловой системой/СУБД) и основным хранилищем, выборочно сохраняя наиболее часто используемые данные на относительно меньшем и более быстром кэширующем устройстве (обычно ОЗУ или SSD). Каждый раз, когда приложение обращается к кэшированным данным, операция ввода-вывода выполняется на кэше, что (обычно) уменьшает время доступа к данным и повышает производительность всей системы [5]. Ниже приведены некоторые общие определения.

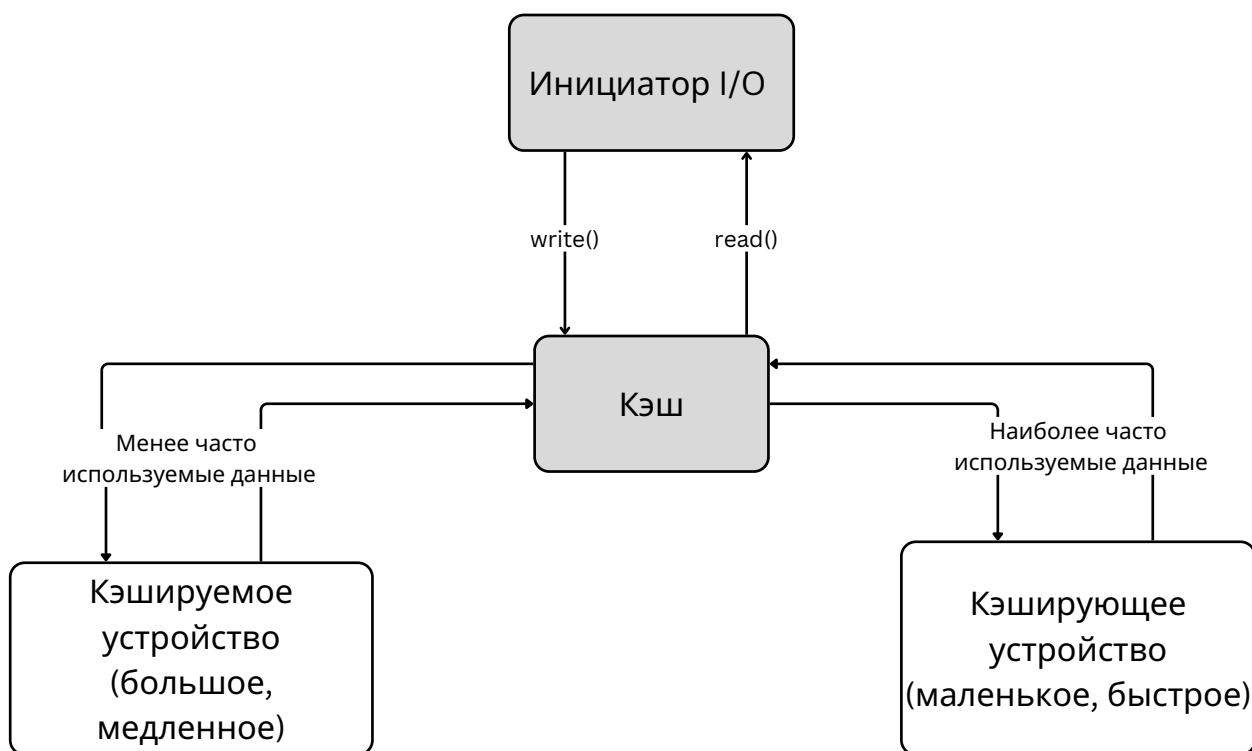


Рис. 1: Базовая схема кэша

Кэш-линия. Минимальная единица хранения данных, отображаемая в кэш. Кэширующее устройство и основное хранилище логически делятся на блоки размера кэш-линии; все отображения выравниваются по этим границам.

Грязная кэш-линия (dirty). Кэш-линия называется *грязной*, если её содержимое новее, чем на основном хранилище, то есть имеются

несброшенные изменения, ожидающие записи на основной носитель.

Чистая кэш-линия (clean). Кэш-линия называется *чистой*, если её содержимое согласовано с данными на основном хранилище и не содержит несброшенных изменений; такую линию можно вытеснить без дополнительной записи на носитель.

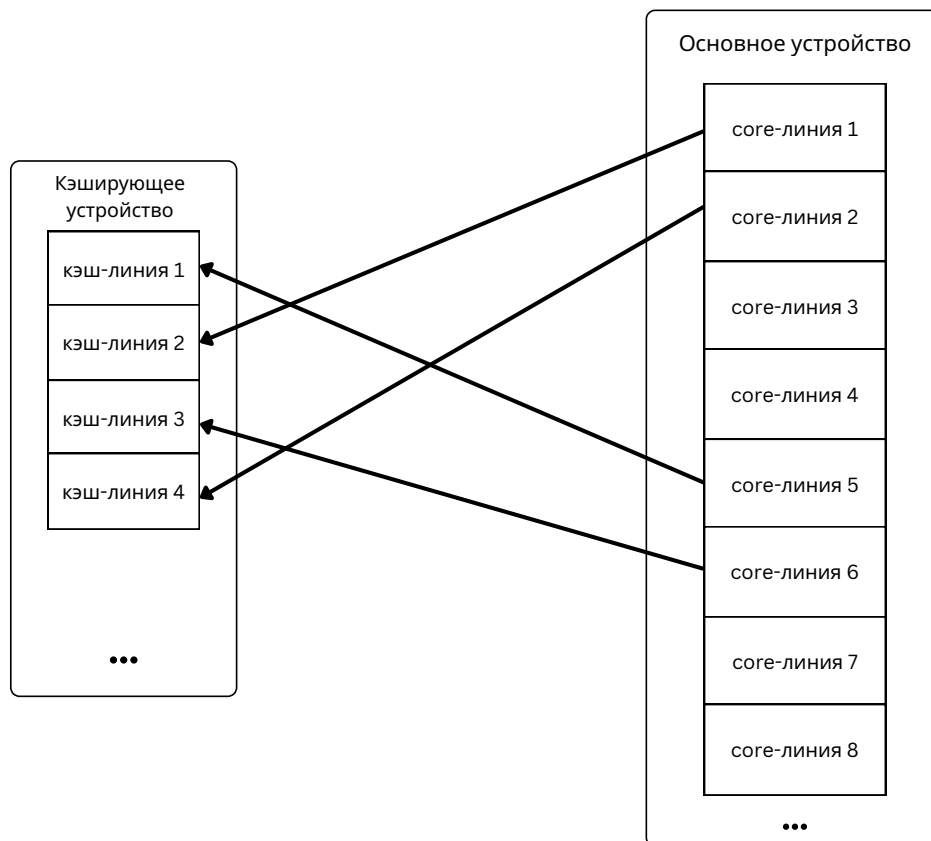


Рис. 2: Отображение блоков основного хранилища в кэш-линии

Доступ к данным определяется **режимом кэширования**:

- *Write-Through* (WT) — запись одновременно производится и в кэш и на основной носитель; ускоряются преимущественно чтения
- *Write-Back* (WB) — запись производится сразу в кэш с последующим асинхронным сбросом; ускоряются чтения и записи (при риске потерь при отказе кэша)
- *Write-Around* (WA) — запись производится на основной носитель; в кэш попадают в основном «горячие» области, снижая загрязнение кэша редкими обращениями

- *Write-Invalidate* (WI) — запись производится на основной носитель, а соответствующие кэш-линии инвалидуются; ускоряются только чтения
- *Write-Only* (WO) — кэш ускоряет только записи; чтения кэш не продвигают
- *Pass-Through* (PT) — обход кэша для всех операций

Политика продвижения (promotion). Фильтр, определяющий, какие обращения и при каких условиях продвигать (вставлять) в кэш, чтобы ограничить загрязнение редкими и последовательными потоками.

Политика вытеснения (eviction). Алгоритм выбора кэш-линий, подлежащих освобождению при нехватке пространства (например, LRU/FIFO/RANDOM, приоритетные очереди и т. п.).

2.1. Блочный уровень ядра Linux

Современные программно-определяемые СХД в серверных средах базируются на Linux и обслуживают нагрузку на уровне блочных устройств; ключевой единицей управления данными становится фиксированный блок. В частности, логика СХД, разрабатываемая компанией **Shvacher**, целиком реализована в пространстве ядра и интегрируется в блочный стек.

Каждая операция над блочным устройством проходит последовательно через несколько подсистем ядра: от уровня VFS и файловой системы к слоям адресации страниц и буферов, далее — в общий блочный слой (Generic Block Layer), планировщик I/O и драйвер устройства вплоть до контроллера накопителя. Ключевой структурой данных на этом пути является дескриптор `struct bio`, описывающий диапазон секторов, направление операции и набор буферов памяти, участвующих в передаче данных. На стороне устройства запросы агрегируются в *request*-объекты и обрабатываются планировщиком с учётом нескольких очередей аппаратного уровня[3].

2.2. Системы кэширования на блочном уровне

Ниже приведён обзор наиболее распространённых систем блочного кэширования, применимых в составе программно-определяемых СХД и совместимых с ядром Linux: *dm-cache* (*LVM Cache*), *bcache*, *Open CAS/OCF*, а также подсистемы *page cache* ядра Linux.

2.2.1. dm-cache (LVM Cache)

Назначение и расположение в стеке. *dm-cache* — *target* инфраструктуры *device-mapper*, интегрированная на блочном уровне между файловыми системами и физическими устройствами. Логическая модель включает *origin* (ускоряемое устройство), *cache* (быстрое устройство, обычно SSD) и *metadata* (персистентные метаданные кэша).

Режимы и политики. Поддерживаются WT/WB/PT. Управление продвижением/вытеснением выполняется подключаемыми политиками; по умолчанию используется стохастическая многоканальная очередь (SMQ), отслеживающая 3 многоуровневые LRU очереди. Очистка «грязных» данных (WB) выполняется асинхронно.

Гранулярность и метаданные. Кэш оперирует линиями фиксированного размера (кратно 32 КиБ). Персистентные метаданные хранятся на отдельном устройстве и отражают соответствия кэш-линий кэширующего и основного устройства, а также состояния *clean/dirty*; предусмотрено восстановление после сбоев[9].

2.2.2. bcache

Идея и архитектура. *Bcache* использует не менее одного быстрого устройства (SSD) как кэш для одного или нескольких «бэкендов» (HDD/объёмные SSD). Отличительная особенность — лог-структурированная организация индекса: ключи диапазонов (*bkey*) в B+дереве, обновления через кольцевой журнал, что повышает устойчивость к сбоям.

Режимы и оптимизации. Поддерживаются WB/WT/WA/PT. Для снижения загрязнения предусмотрен *sequential cutoff*: длинные последовательные запросы направляются в обход кэша.

Гранулярность, GC и метаданные. Единица аллокации — *bucket* (десятки/сотни КиБ до МиБ). Для бакетов поддерживаются приоритеты (LRU), поколения (инвалидация устаревших версий), флаги загрязненности. Сборщик мусора координируется с аллокатором и потоками записи; восстановление после сбоев выполняется воспроизведением журнала [8].

2.2.3. Open CAS / OCF

Архитектурные основы. *Open Cache Acceleration Software (CAS)* базируется на библиотеке *Open Caching Framework (OCF)* и реализует блочный кэш, где базовой единицей выступает *кэш-линия*. Каждая кэш-линия сопоставляется *core*-линии на бэкенде; в метаданных поддерживаются посекторные (*per-sector*) флаги *clean/dirty*.

Режимы и политики. Доступны режимы: WT, WB, WA, WI, WO, PT. Прогнозирование управляется политикой *nhit*, фильтрующей шумовые обращения. Предусмотрены 2 алгоритма вытеснения неиспользуемых данных: LRU и *Aggressive Cleaning Policy (ACP)*, вытесняющая блоки состоящие из нескольких кэш-линий за раз. Используется *sequential cutoff* для последовательных запроса.

Метаданные и целостность. Метаданные сегментированы по ролям (карты, LRU, служебные структуры), хранятся на кэш-устройстве; обеспечиваются контроль целостности и процедуры восстановления [5].

2.2.4. Подсистема page cache ядра Linux

Роль в едином пути I/O. *Page cache* — общий кэш страниц, через который проходят обычные чтения/записи и. Он повышает эффективность повторных чтений и сглаживает интенсивность записей за счёт

отложенного сброса «грязных» страниц и *readahead*. Наряду с данными файлов виртуальная файловая система кэширует собственные структуры, сокращая накладные расходы на разрешение путей и доступ к метаданным. Для высоконагруженных приложений возможно использование `O_DIRECT`, исключающего буферизацию в page cache.

Ограничения при работе с метаданными СХД. Управление page cache осуществляется общей подсистемой виртуальной памяти и ориентировано на файловые страницы, а не на специфичные структуры СХД. Это ограничивает контроль над *layout*-ом метаданных в ОЗУ, приоритизацией и порядком вытеснения объектов [6], [3].

2.3. Выводы

Рассмотренные системы решают близкую, но отличную от целей данной работы задачу: Open CAS, dm-cache и bcache предназначены для SSD кэширования, page cache может быть подключен для кэширования метаданных в оперативной памяти, но он не предоставляет управления размещением памяти, не позволяет интегрировать алгоритмы отличные от уже реализованных без существенной переработки исходного кода ядра Linux.

3. Описание решения

В исходной системе все метаданные хранятся одновременно в RAM и на диске (meta-томах), что приемлемо для небольших объемов, но становится неустойчиво для «тяжелых» подсистем — лог-структурированного маппинга и таблиц дедупликации, чьи размеры велики и растут вместе со сроком жизни тома. Чтобы снять давление на оперативную память, было принято решение ввести управляемый RAM-кэш метаданных с подгрузкой «по требованию» и вытеснением неактивных данных. Первая итерация ориентирована на лог-структурированный маппинг, однако архитектура сразу закладывает расширяемость на другие типы мета-томов.

3.1. Внесенные изменения

Каждый том, управляемый движком СХД, обладает массивом `cell_table`, содержащим структуры `ori_cell_info`, ответственные за организацию лог-структурированности. Для реализации новой функциональности, массив был заменён на обертку над контроллером кэша `struct ori_cell_info_cache`, оперирующей кэш-линиями размером с 1 страницу памяти (4 КиБ). Кэш-линии `ori_cell_info_cl` содержат в себе массив `cell_info` и структуру `ori_mc_entry`, хранящую информацию о состоянии кэш-линии.

Основным компонентом системы является контроллер кэша `struct ori_mc_ctl`. Контроллер не имеет никакого представления о хранимых данных, а оперируют лишь состояниями `ori_mc_entry`, что накладывает на обертку требование реализовать логику формирования запросов чтения/записи самостоятельно. При необходимости сброса на диск, или, наоборот, чтения, контроллер обращается к необходимым функциям через структуру `ori_mc_ops`.

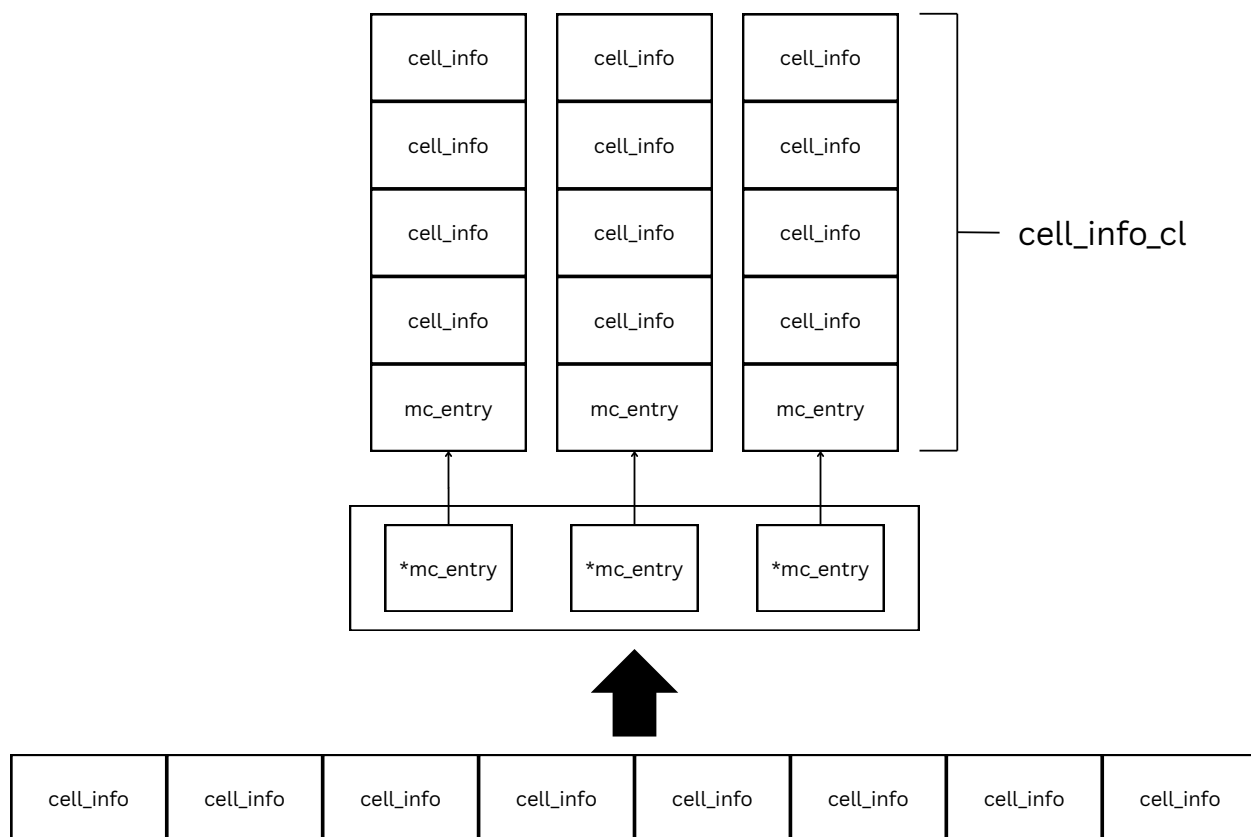


Рис. 3: Реорганизация хранения `cell_info`

Все единицы кэша `ori_mc_entry`, в зависимости от состояния, могут быть помещены в один из двух LRU списков: `evictable` и `dirty`, сами списки объединены в структуру `ori_mc_lru` и защищены спинлоком для упрощения синхронизации.

3.2. Жизненный цикл кэш-линии

Жизненный цикл кэш-линии реализован при помощи следующих состояний:

- **NONE** — кэш-линия не представлена в оперативной памяти
- **LOADING** — кэш-линия подгружается в оперативную память с диска
- **EVICTABLE** — кэш-линия чистая и никто не держит на неё ссылку, т.е. она может быть вытеснена без предварительной записи содержимого на диск

- UPTODATE — кэш-линия чистая, но кто-то держит на неё ссылку
- DIRTY — кэш-линия грязная, не может быть вытеснена без предварительного сброса

Когда к линии обращаются впервые, она «оживает» из NONE — переходит в LOADING, читается с диска и становится UPTODATE. До тех пор пока на неё есть ссылки, она остаётся UPTODATE; когда счётчик ссылок опускается до нуля линия переводится в состояние EVICTABLE. В этом состоянии линия при необходимости может стать жертвой вытеснения, что возвращает её обратно в NONE. Любой новый доступ к EVICTABLE снова делает линию UPTODATE. Если в процессе обработки данные линии были модифицированы, она помечается как DIRTY, в последствии writeback синхронизирует данные с основным хранилищем и вернет линию в UPTODATE (или сразу в EVICTABLE, если к этому моменту ссылок нет).

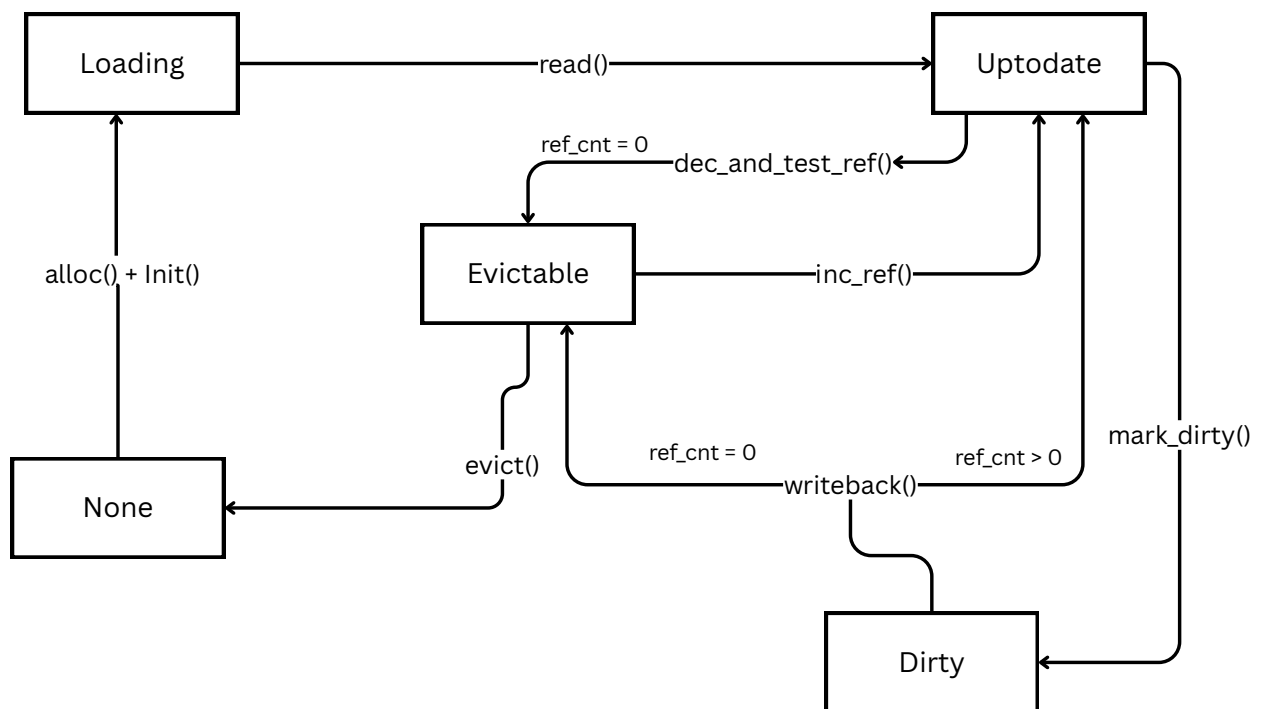


Рис. 4: жизненный цикл кэш-линии

3.3. Политика вытеснения

На данный момент используется простая LRU политика с двумя очередями: `evictable` (чистые, свободные линии) и `dirty` (линии с несброшенными изменениями). Обе очереди — двусвязные списки ядра под общим спинлоком `lru.lock`. Пометка на запись (`ori_mc_entry_mark_dirty`) переводит линию в состояние DIRTY и переносит её в голову `dirty`; `writeback` обходит `dirty` с хвоста, сбрасывает кандидатов с `refcnt=0` и возвращает их в пул чистых. Чистые линии без ссылок (`refcnt=0`) находятся в `evictable` и при давлении на память выбираются оттуда для вытеснения.

3.4. Обработка поступающих запросов

Входящие обращения к метаданным идут через обёртку `ori_cell_info_cache`, которая перенаправляет запросы к контроллеру кэша, и сводятся к трём действиям: взять линию (`ori_mc_entry_get`), при необходимости пометить её как изменённую (`ori_mc_entry_mark_dirty`) и вернуть (`ori_mc_entry_put`).

На данный момент, для упрощения реализации, во время обработки линия блокируется per-entry замком. Для извлечения и модификации данных, хранящихся в кэш-линии, применяются специальные `get/set` функции.

Взятие линии. Если линия уже лежит в оперативной памяти:

- для UPTODATE просто увеличивается счётчик ссылок (ввиду блокирования линии на данный момент невозможно);
- для EVICTABLE линия снимается со списка `evictable` и переводится в UPTODATE;
- для DIRTY ссылка увеличивается, а сама линия продвигается в начало списка `dirty`.

Если линии в оперативной памяти нет, при полной заполненности кэша запускается вытеснение: из списка `evictable` берется последняя кэш-линия, если список пуст запускается сброс грязных линий (`writeback`) и попытка повторяется. После этого новая линия аллоцируется из слаба, содержимое подгружается с диска через `ops->read`, помечается как `UPTODATE` и учитывается в счётчике объектов.

Пометка изменения. Set функции обертки `ori_cell_info_cache` обязаны пометать кэш-линии как грязные при модификации содержимого. Дальнейшее состояние линии управляется контроллером.

Возврат линии. Счетчик ссылок на кэш-линию уменьшается и снимается блокировка. Когда счётчик опускается до нуля, `UPTODATE` линии отправляются в `evictable` и становятся кандидатами на вытеснение. «Грязные» при нулевых ссылках остаются в `dirty` до сброса — вызывать `writeback` сразу нельзя из-за LRU политики.

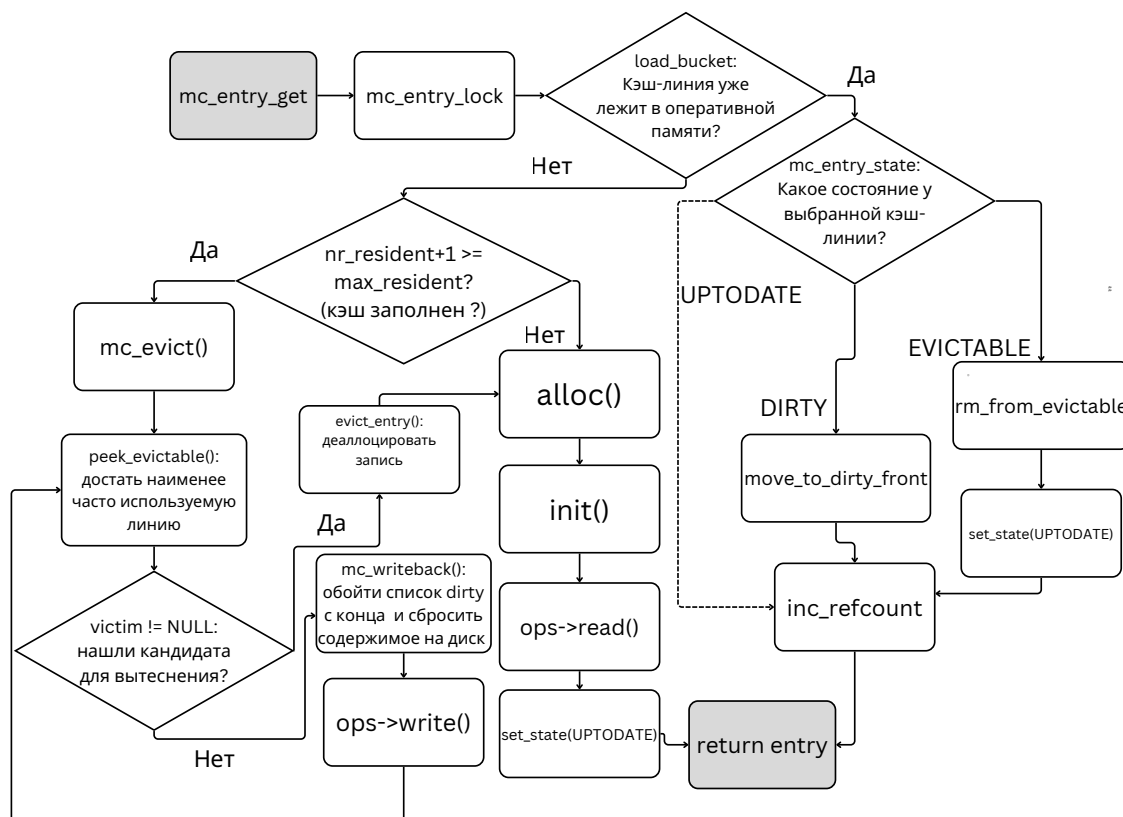


Рис. 5: Граф потока управления взятия линии `ori_mc_entry_get()`

4. Тестирование

Тестирование системы проводилось на наборе томов различного размера. Для инициализации тестового окружения использовалась внутренняя утилита `oridevctl`, позволяющая управлять дисковой подсистемой. Подготовка тестового окружения производилась следующим образом:

```
# Добавление физических дисков в систему
```

```
./oridevctl drive_add --drive_name=d1 --drive_path=/dev/sdb  
./oridevctl drive_add --drive_name=d2 --drive_path=/dev/sdc  
./oridevctl drive_add --drive_name=d3 --drive_path=/dev/sde
```

```
# Очистка конфигурации
```

```
./oridevctl drive_clear --all
```

```
# Создание пула устройств
```

```
./oridevctl pool_create --pool_name=p1 --drive_count=3 \  
--drive_names=d1,d2,d3
```

```
# Создание томов различного размера для тестирования
```

```
./oridevctl vol_create --vol_name=vol1 --pool_name=p1 \  
--size_mb=5120 --ls=1  
./oridevctl vol_create --vol_name=vol2 --pool_name=p1 \  
--size_mb=10240 --ls=1  
./oridevctl vol_create --vol_name=vol3 --pool_name=p1 \  
--size_mb=20480 --ls=1
```

Для проверки корректности работы системы проведено функциональное тестирование на томах различного размера (5ГБ, 10ГБ и 20ГБ). Основной целью тестирования являлась проверка целостности данных при операциях записи и чтения, а также проверка работы системы кэширования при различных размерах блоков данных и размерах томов.

Для тестирования использовалась утилита Flexible I/O Tester

(fio) [2] — современный инструмент для тестирования подсистем ввода-вывода. FIO позволяет генерировать различные паттерны нагрузки и проводить верификацию записанных данных. Тестирование проводилось в два этапа. На первом этапе выполнялась запись данных с различными размерами блоков:

```
fio --name=ls --ioengine=io_uring --iodepth=16 --rw=write \  
    --bssplit=4k/30:8k/30:16k/30:512k/10 --direct=1 --numjobs=1 \  
    --size=5G --filename=/dev/ori-vol1 --verify=sha256\  
    --do_verify=0 --verify_fatal=0 --verify_only=0
```

На втором этапе производилось чтение записанных данных с проверкой их целостности:

```
fio --name=ls --ioengine=io_uring --iodepth=16 --rw=read \  
    --bssplit=4k/30:8k/30:16k/30:512k/10 --direct=1 --numjobs=1 \  
    --size=5G --filename=/dev/ori-vol1 --verify=sha256 \  
    --do_verify=1 --verify_fatal=1 --verify_only=1
```

Использованные параметры тестирования:

- `ioengine=io_uring` — использование современного асинхронного интерфейса ввода-вывода `io_uring`
- `iodepth=16` — количество параллельных операций ввода-вывода
- `bssplit` — распределение размеров блоков: 30% операций с блоками 4KB, 8KB и 16KB, 10% с блоками 512KB
- `direct=1` — использование прямого ввода-вывода в обход системного кэша
- `verify=sha256` — проверка целостности данных с использованием SHA-256

В результате проведенного тестирования корроупции данных не было выявлено: все записанные блоки были успешно прочитаны и верифицированы с помощью SHA-256 хеш-сумм. Это подтверждает корректность работы системы кэширования при различных размерах блоков и томов.

Заключение

В результате данной работы были достигнуты следующие результаты:

- Проведен обзор существующих решений.
- Разработан прототип системы кэширования метаданных.
- Прототип интегрирован, проверено соответствие ожидаемому поведению и отсутствие нарушения существующей логики.

Код является проприетарным и используется в рамках внутренней разработки компании Shvacher.

В дальнейшем планируется оптимизировать производительность системы путем реализации lock-free LRU политики, а также расширить функциональность, добавив поддержку кэширования для других типов метаданных.

Список литературы

- [1] Apache Cassandra Architecture. — URL: <https://cassandra.apache.org/doc/latest/cassandra/architecture/architecture.html> (дата обращения: 10 сентября 2025 г.).
- [2] Axboe Jens. — Flexible I/O Tester (fio) documentation. — fio project, 2024. — Version 3.38. URL: https://fio.readthedocs.io/en/latest/fio_doc.html (дата обращения: 10 июня 2024 г.).
- [3] Bovet Daniel, Cesati Marco. Understanding The Linux Kernel. — O'Reilly & Associates Inc, 2005. — ISBN: 0596005652.
- [4] F2FS: A New File System for Flash Storage. — URL: <https://www.kernel.org/doc/html/latest/filesystems/f2fs.html> (дата обращения: 10 сентября 2025 г.).
- [5] Open CAS documentation. — URL: <https://open-cas.com/index.html> (дата обращения: 10 сентября 2025 г.).
- [6] Overview of the Linux Virtual File System. — URL: <https://www.kernel.org/doc/html/next/filesystems/vfs.html> (дата обращения: 10 сентября 2025 г.).
- [7] Rosenblum Mendel, Ousterhout John K. The design and implementation of a log-structured file system // *ACM Trans. Comput. Syst.* — 1992. — Vol. 10, no. 1. — P. 26–52. — URL: <https://doi.org/10.1145/146941.146943>.
- [8] bcache. — URL: <https://www.kernel.org/doc/Documentation/bcache.txt> (дата обращения: 10 сентября 2025 г.).
- [9] dm-cache. — URL: <https://www.kernel.org/doc/Documentation/device-mapper/cache.txt> (дата обращения: 10 сентября 2025 г.).